ORCHARDVISION · PLAN v1.3 · 2 OCTOBER 2026

Complete Development Plan

A ground-control and orchard-intelligence platform that plans a drone survey, flies it on a virtual or real drone, and turns the footage into a tree map and a fruit count with a stated uncertainty.

1. What we are building
2. The drone software
3. Comparison with other ground stations
4. Orchards, crops and models
5. How the camera captures
6. Hidden fruit: the three ideas, checked
7. Simulation and the virtual flight
8. Step-by-step development
9. Things noticed while planning
10. Decisions still open

Companion files: PLAN.md (the locked contract) and PRIOR_WORK.md (the literature scan with links).

1. What we are building

OrchardVision is one application with three jobs. It is a **ground station** that plans and supervises a drone flight. It is an **analysis system** that finds every tree and counts the fruit on it. It is a **simulator** that flies the same mission on a virtual drone over a virtual orchard, so the whole chain can be tested before any hardware exists.

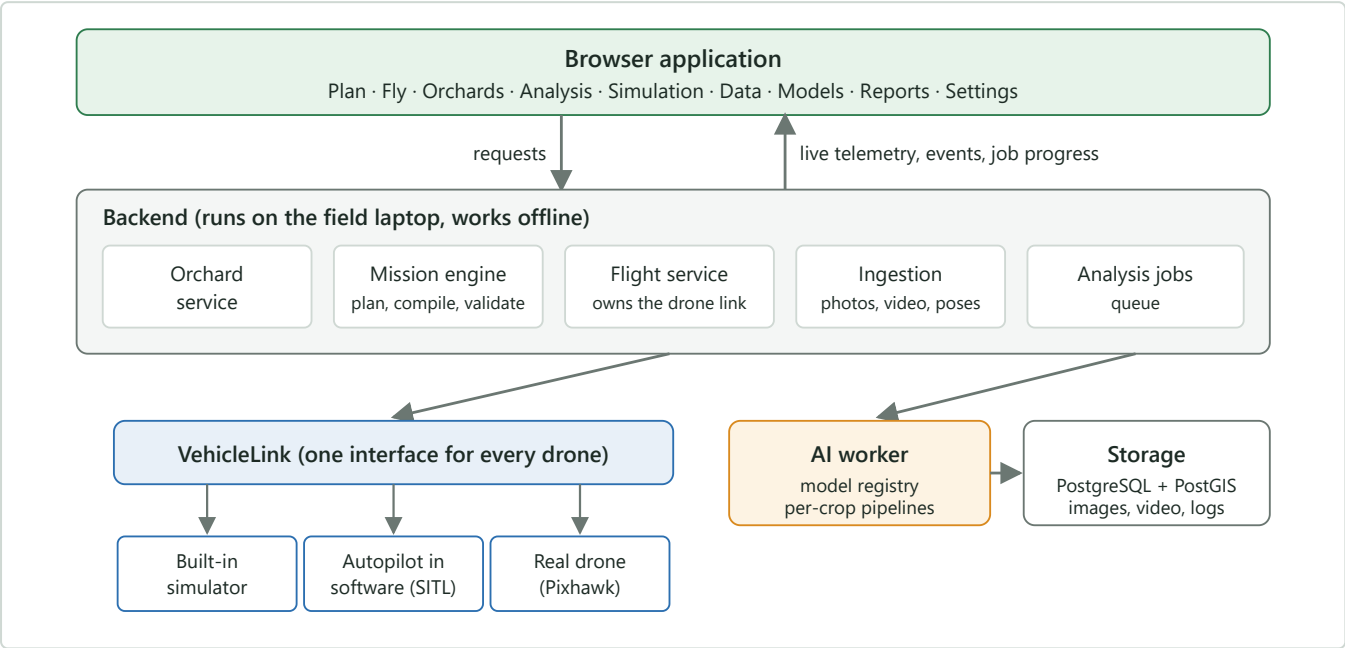


Figure 1. The parts of OrchardVision. The flight service is the only part that talks to a drone, and it does so through one interface whether the drone is virtual or real.

The four rules every phase follows

1. **One mission, three modes.** The same mission is previewed, simulated and flown. Nothing is planned separately for the simulator.
2. **One vehicle interface.** The software talks to a virtual drone and a real drone in exactly the same way.
3. **One data pipeline.** A simulated capture and a real capture enter through the same door in the same format.
4. **Crop is data.** Papaya, mango and apple differ in a crop profile, never in program code.

What the user does, start to finish

1. Create an orchard, choose its crop and draw its boundary on the map.
2. Generate a high overhead survey, check it, and fly it (virtual or real).
3. The overhead photos become a tree map: every tree gets a permanent ID and a position.
4. The tree map generates a second, low mission that films the fruit on chosen trees.
5. The video becomes a fruit count per tree, with a range that says how sure the system is.
6. Results appear on the dashboard and in a report.

2. The drone software

This is the ground-station part: everything needed to plan a flight, supervise it, and keep a record of it.

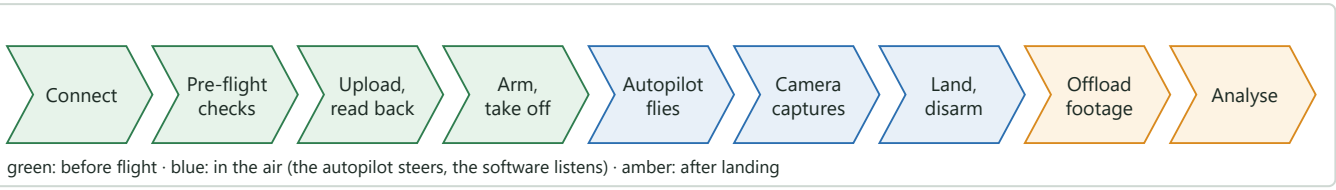


Figure 2. One flight from connection to analysis. During the flight the autopilot navigates by itself; OrchardVision does not steer from point to point.

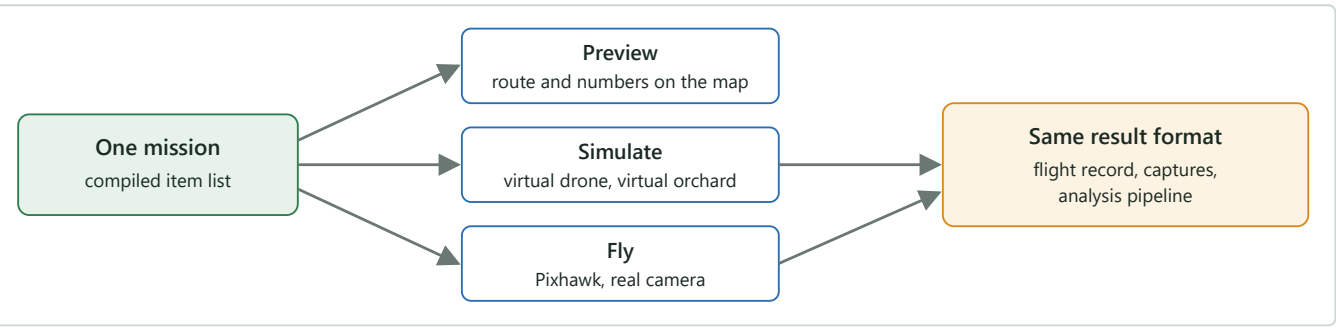


Figure 3. The mission is written once. Simulation and real flight execute the identical item list and produce records in the identical format.

The table lists every function we build and how it works.

Function	How it works
Connect	The flight service opens a link: in-process for the built-in simulator, a network port for the software autopilot, a telemetry radio or the on-board computer's network link for the real drone. The drone sends a heartbeat every second; three missed heartbeats mark the link as lost.
Live telemetry	Every incoming message is turned into one common record: position, height, speed, attitude, heading, battery, GPS quality, flight mode, armed state, current mission item. The record is pushed to the browser about five times a second.
Pre-flight checklist	Computed, not ticked by hand. GPS fix, satellites, battery, home position and estimator health come from telemetry. Mission validated, model loaded and storage free come from the software. Weather and visual inspection are manual confirmations.
Mission planner	Three mission types: overhead survey, fruit observation, custom waypoints. The user sets intent (area, height, overlap, speed); the mission engine compiles it into a flat list of items: take off, go to point, set speed, start and stop camera, tilt camera, return, land.
Survey generator	Takes the orchard boundary and the camera profile. Computes how much ground one photo covers, spaces parallel flight lines so neighbouring photos overlap, places turnarounds outside the boundary, and sets the distance between photos.
Fruit-mission generator	Takes the stored tree map and the crop's fruit-pass geometry. Picks the trees to observe (all, or a statistical sample), then lays a path along the gaps between rows so each chosen tree is filmed from both sides.

Mission validation	Checks geometry, that every item is inside the fence, height and speed limits, that image detail meets the crop's minimum, flight time and battery against the reserve, storage needed, a return path from every point, and that the crop's models exist. A mission too long for one battery is split into segments that resume at a known item.
Upload and read-back	The item list, fence and failsafe settings are sent to the autopilot. The software then reads everything back and compares it with what it sent. Any difference blocks the flight.
Live flight view	Map with planned route and actual track, height, speed, battery, GPS, attitude, mission progress, capture count. The same view serves virtual and real flights, with a badge showing which.
Commands	Arm, take off, start, pause, resume, return home, land. Each waits for the autopilot's acknowledgement and shows a failure if none arrives.
Safety settings	Height and speed limits, fence, battery thresholds, and the action on link loss or GPS loss. These are uploaded to the autopilot, which carries them out on its own. OrchardVision configures, verifies and displays them.
Flight record and replay	Each flight stores its mission revision, telemetry, events and captures. Any flight can be replayed on the map.

3. Comparison with other ground stations

QGroundControl and Mission Planner are the two established open ground stations. They are general tools for any drone and any job. OrchardVision is narrower on flying and far deeper on orchards.

Capability	QGroundControl	Mission Planner	OrchardVision
Firmware install, sensor calibration, tuning	Yes	Yes	No, use one of the others
Waypoint missions	Yes	Yes	Yes, basic
Area survey with photo overlap	Yes	Yes	Yes
Corridor and structure scans	Yes	Partial	No, export instead
Geofence	Yes	Yes	Yes
Rally point editing	Yes	Yes	No in V1
Live map, telemetry, commands	Yes	Yes	Yes, the essentials
Joystick flying, live video	Yes	Yes	No in V1
Deep flight-log analysis	Partial	Yes	No, replay only
Several drones at once	Yes	Partial	No
Runs in a browser	No	No	Yes
Orchards, crops and trees as stored records	No	No	Yes
Mission generated from a tree map	No	No	Yes
Validation that blocks: battery, image detail, model present	Shows statistics	Shows statistics	Yes, with flight splitting
Upload read-back comparison	Partial	Partial	Yes, blocks on mismatch
Built-in virtual drone	Connects to one	Launches SITL	Yes, two kinds
Virtual orchard with known fruit counts	No	No	Yes
Matching photos to capture positions	Geotag tool	Geotag tool	Automatic, with quality checks
Video with per-frame camera pose	No	No	Yes
Tree detection, tree map, permanent tree IDs	No	No	Yes
Fruit count per tree with uncertainty	No	No	Yes
Analytics and reports	No	No	Yes

What is extra in OrchardVision

- **It knows what an orchard is.** Orchards, crops, trees, surveys and estimates are stored records, not just lines on a map.

- **The analysis writes the next mission.** The tree map from flight one generates the fruit mission for flight two.
- **Simulation is a test bench, not a demo.** The virtual orchard knows the true fruit count of every tree, so the whole pipeline is scored automatically.
- **Validation understands the crop.** It refuses a mission whose photos would be too coarse for the crop's model.
- **Flight and analysis share one record.** Every result traces back to the frame, the camera pose, the flight, the mission revision and the model version.

> **What we deliberately do not build.** Firmware installation, sensor calibration, tuning, joystick control and deep log analysis already work well in QGroundControl and Mission Planner. One of them stays installed for drone setup. Our missions export to the QGroundControl plan format, so a mission can always be opened there.

4. Orchards, crops and models

Papaya is the first crop, not the only one. The platform is written once. Everything that differs between crops lives in a **crop profile**, a folder of files that the software reads.

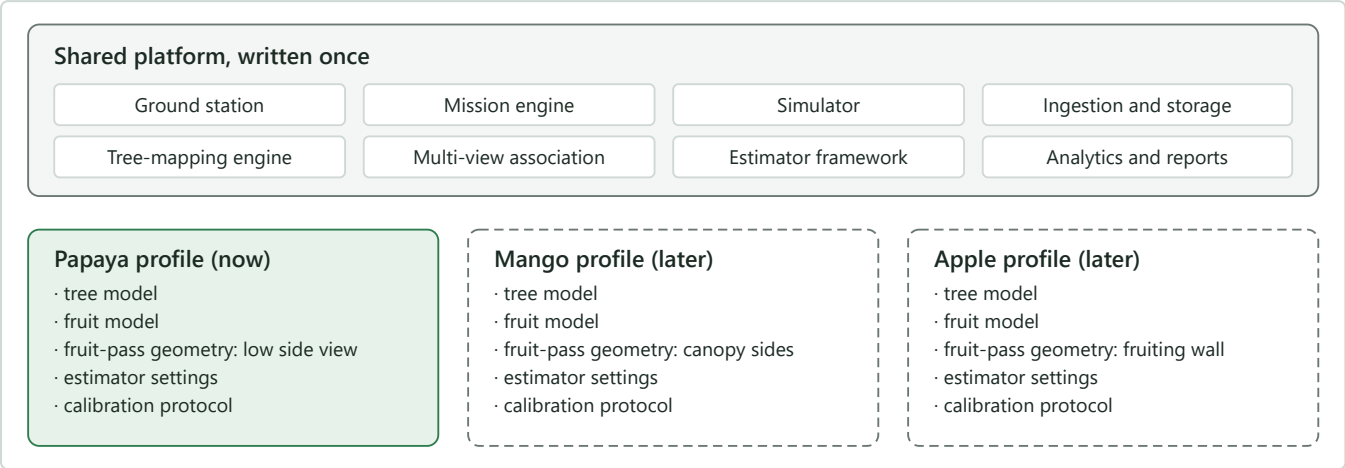


Figure 4. A crop is a profile plugged into the shared platform. Adding a crop means adding a profile, not changing the program.

What a crop profile contains

Item	What it holds	Papaya example
Tree model	Weights that find tree crowns in overhead photos, plus a manifest with checksum, input size and test scores	papaya tree model, version 1
Fruit model	Weights that find fruit in side-view frames, with the same manifest	papaya fruit model, version 1
Crop facts	Typical tree spacing, crown size, fruit size, the coarsest image detail the models accept	spacing about 2 m, oblong fruit
Fruit-pass geometry	Where the camera must be to see fruit: height, distance, tilt, number of sides, speed	low, level, both sides of the row
Estimator settings	Which correction steps to apply and their starting values	trunk-column model
Calibration protocol	How many trees to hand-count and how to choose them	about 18, spread by crown size

How the right models get loaded

1. The user opens an orchard. The orchard record names its crop.
2. The model registry looks up the crop's profile and its active model versions.
3. The worker checks each weights file against its checksum and loads it.
4. Every result stores the model version that produced it, so old results stay explainable after a model is replaced.

Steps to add a new crop later

1. Run the viewpoint study for that crop (section 8, Phase 0) to learn where its fruit can be seen from.
2. Collect and label images; train a tree model and a fruit model.

3. Write the profile files and register the model versions.
4. Build a virtual version of the crop for the simulator so the pipeline can be scored.
5. Hand-count calibration trees in a real orchard and check the estimates against them.

> **Honest limit.** Tree mapping carries over between crops almost unchanged. The fruit pass does not: fruit hang on a papaya trunk, throughout a mango canopy, and on a flat apple wall. Each crop needs its own capture geometry and its own field check. The platform makes that a configuration task, but it is still real work per crop.

How the four planned crops differ

Crop	Plant shape	Where the fruit is	Fruit pass needed	Main difficulty
Papaya	One trunk, small crown on top	In a column on the trunk under the crown	One low side pass, both sides of the row	Green fruit on green; fruit behind fruit
Mango	Large round canopy, trees well apart	Around the outside of the canopy and inside it	Side passes at two or three heights, all round	Much fruit inside the foliage; tall trees
Orange	Dense round canopy, often touching in rows	Throughout the canopy	Side passes plus an angled view from above	High hidden share; trees merge into hedges
Apple	Thin flat wall on a trellis	Spread over the wall	Passes along the wall at two or three heights	Trees not separable; hail nets block drones

Readiness audit: where papaya was baked in, and the fix

The crop-profile idea was right, but five parts of the plan quietly assumed papaya. Each is now general in the contract, while only the papaya version is built in V1.

Part	Papaya assumption	Now
Fruit-mission generator	One low path along the row gaps	A viewpoint template from the profile: pass type, list of heights, distance, tilt, sides
What gets counted	Always a single tree; fruit belongs to the nearest trunk	A counting unit: a tree, or a stretch of row for hedges and walls. The assignment rule comes from the profile
Estimator	Hidden fruit are behind other fruit	Takes tree measurements (height, crown width, canopy volume) as inputs, because in dense canopies the hidden share depends on canopy size
Sensors	One colour camera	Each capture belongs to a sensor stream, so a depth camera or LiDAR can be added later without changing stored data
Simulator	Trunk with a spiral of fruit	Plant generator chosen by the profile: trunk column, round canopy, or wall

3D for every crop

- **Height model from Pass A (built in Phase 7).** The overhead photos are also processed into a stitched map and a surface height model. This gives each tree's height, crown width and canopy volume, which the estimator uses and which other crops need most.

- **Schematic 3D view (Phases 11 to 13).** Each tree or row segment is drawn from measurements: the plant shape for its crop, with every fruit at its measured position. Observed fruit are solid; estimated fruit are drawn differently. The crown can be hidden to show the fruit.
- **After V1.** Photoreal reconstruction of selected trees, and LiDAR for dense-canopy crops where it earns its cost. Neither helps papaya counting much.

> **Paper test before the contracts freeze.** At Milestone A, one dense-canopy crop (mango is recommended, as the opposite of papaya) is written up as a profile and walked through every contract. No code is built for it. If the mango profile fits without changing a contract, the platform is ready for other fruit.

5. How the camera captures

One flight cannot do both jobs. From high above, the leaf crown of a papaya hides the fruit, which hang on the trunk underneath. So each survey has two passes.

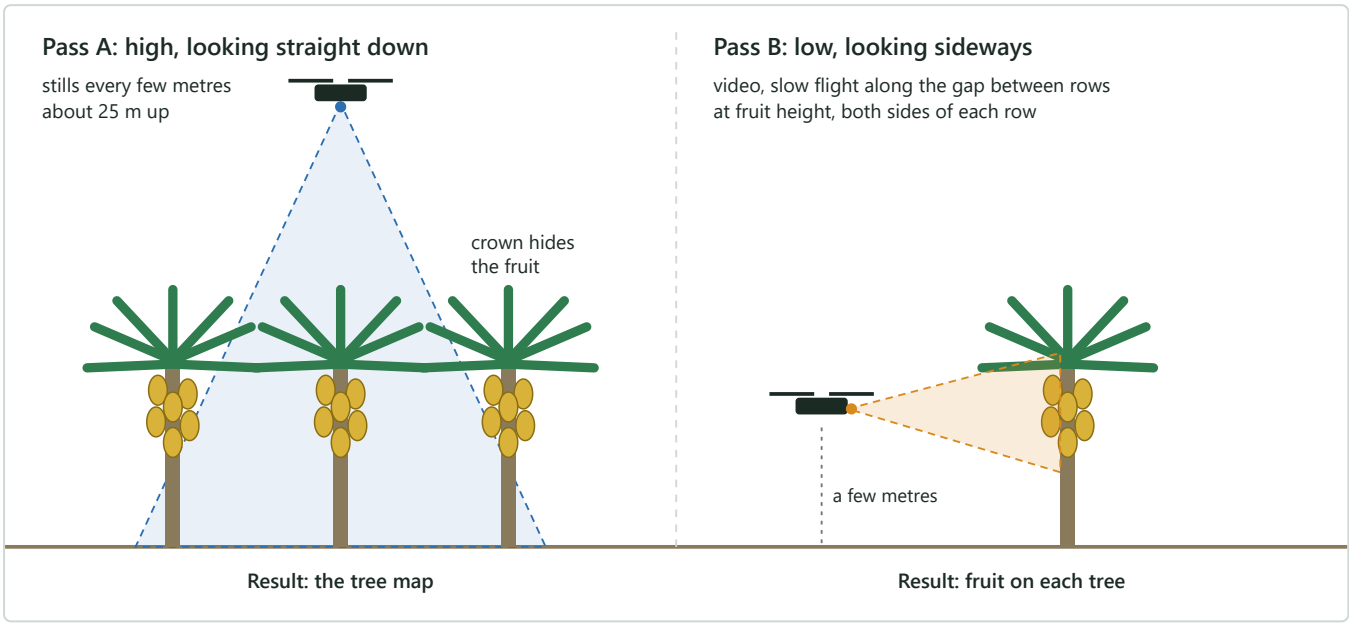


Figure 5. The two passes, not to scale. The exact height, distance and tilt of Pass B are chosen from the Phase 0 field study, because nobody has published them for papaya.

Pass A: overhead survey

- **What triggers the camera:** the autopilot, every fixed distance along each survey line. Each trigger produces a capture record: time, position, height, heading, camera tilt.
- **How spacing is calculated:** from the camera profile.

  `image detail = sensor width × height ÷ (focal length × image width)`

  `line spacing = ground width of a photo × (1 − side overlap)`

  `photo distance = ground length of a photo × (1 − front overlap)`
- **Worked example** for a typical one-inch camera (13.2 mm sensor, 8.8 mm lens, 5472 × 3648 pixels) at 25 m: each pixel covers 0.69 cm, one photo covers 37.5 m × 25 m, lines are 11.3 m apart at 70% side overlap, and a photo is taken every 5 m at 80% front overlap. At 3 m/s that is one photo every 1.7 seconds.
- **Height is tested, not assumed.** In published banana work, detection fell from 96% at 40 m to 76% at 60 m. We test two or three heights and keep the best.

Pass B: fruit observation

- **Video, not stills.** Linking the same fruit between views needs frames that overlap heavily. Published trackers use about ten frames a second.
- **Pose for every frame.** The on-board computer records video with timestamps and logs the drone's position and attitude many times a second. After landing, each frame gets a pose by interpolating the log at the frame's time.
- **Both sides of each row.** The drone flies the gap on one side of a row, then the gap on the other side. As it passes a tree the viewing direction sweeps round the trunk.

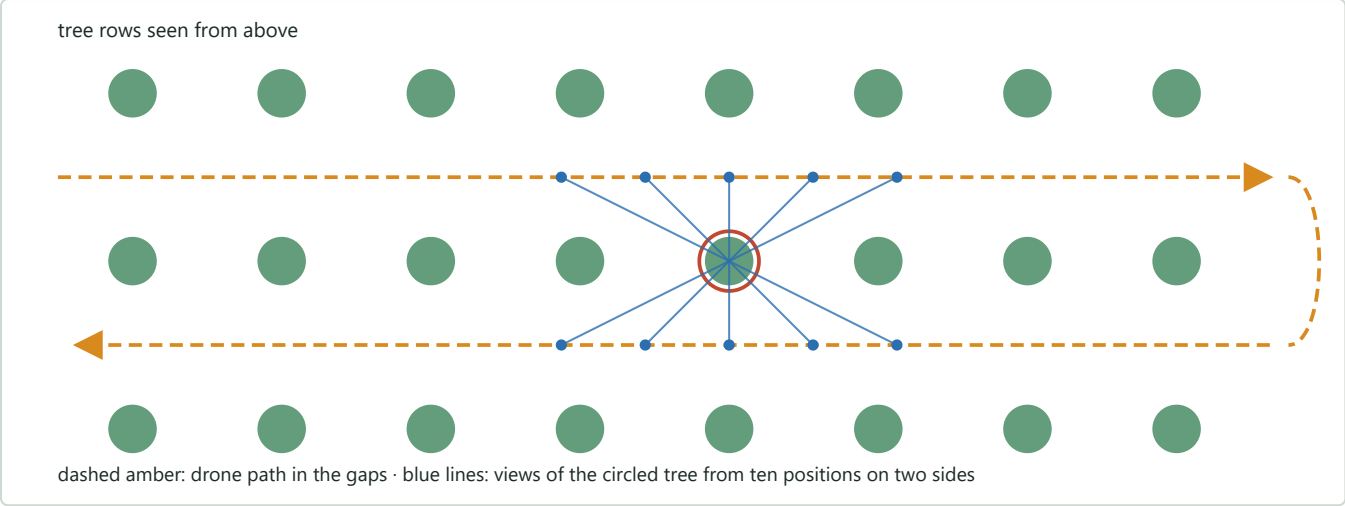


Figure 6. The fruit pass from above. One tree is seen from many directions without the drone ever circling it.

Camera requirements this creates

- **Fast shutter.** At about 2 m from the trunk and 1 m/s, the shutter must be around 1/500 s or faster to avoid blur.
- **Fixed focus and fixed exposure mode** so the fruit model sees consistent images.
- **Clock agreement** between camera and autopilot. At 1 m/s, a 50 millisecond error moves a frame's position by 5 cm.
- **Looking up into the sky** under the crown makes fruit dark against bright light; time of day and exposure matter and are recorded with each flight.

> **Flight time finding.** With rows 2.5 m apart, a two-hectare orchard has about 8 km of row. Filming every row from both sides at 1 m/s is roughly 4.5 hours of flying with one side-facing camera, or 2.2 hours with two. That is many batteries. The plan therefore supports two modes: **full coverage** for small blocks, and **sampled coverage**, where the fruit pass films a statistical sample of trees and the tree map (which counts every tree) scales the result to the orchard. Published sampling work needs between 20 and 156 trees for a 90% range within 10%.

6. Hidden fruit: the three ideas, checked

You proposed three ideas: photograph from different directions, complete the shape of half-visible fruit, and use a statistical model. All three belong in the plan, but each solves a different kind of "hidden".

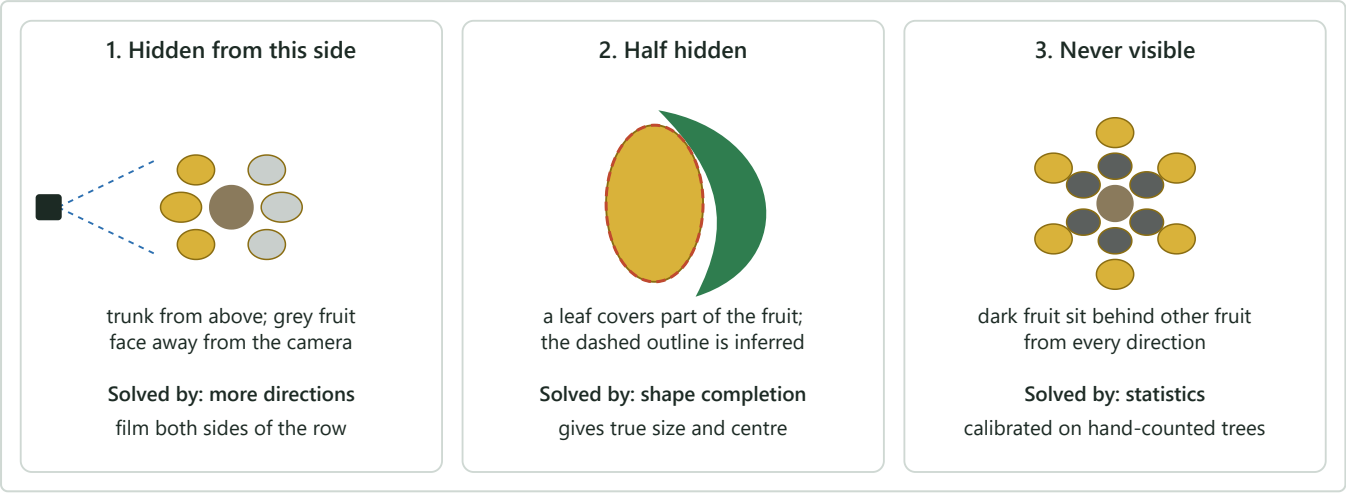


Figure 7. Three kinds of hidden fruit and the idea that addresses each.

Idea 1: photographs from different directions — yes, this is the main method

- **Evidence.** In mango, video tracking along the row saw 62% of the harvested fruit, against 40% from two still views. A multi-view mango system reached 1.36% error per tree without any correction factor.
- **How it works here.** The fruit model finds fruit in every frame. Because each frame has a known camera position, a fruit seen in two frames can be placed in 3D where the two sight lines cross. Detections that land on the same 3D point are one fruit. This removes double counting and merges both sides of the row.
- **Why it suits papaya.** The trunk is a single column with no branches, so each fruit has a clear position: height on the trunk and angle around it.

Idea 2: completing the shape of half-visible fruit — useful, but not for counting

- **What it really gives.** A fruit that is half visible is already detected and counted as one. Completing its outline does not find fruit that are fully hidden.
- **Where it helps.** It gives the true size (so yield can be stated in kilograms, not only in fruit), a better centre point (so matching between views is more reliable), and cleaner separation of fruit that overlap.
- **Evidence.** Apple work that predicts the full outline of occluded fruit measured diameter to within 4.5 mm on average, and 2.9 mm when restricted to fruit more than 60% visible.
- **In the plan.** A supporting step inside Phase 11. Papaya fruit are oblong, so the completed outline is fitted with an ellipse. It is built after the basic count works, not before.

Idea 3: a statistical model — yes, for what no camera can see

The estimate is built in layers, and every layer is reported so nothing is hidden inside one number.

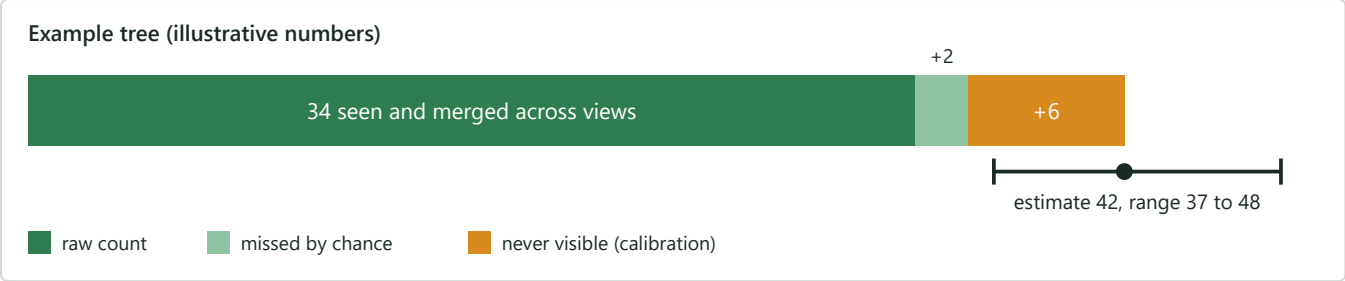


Figure 8. The layers of one tree's estimate.

Layer	How it is calculated
1. Raw count	Unique fruit after merging all views. This is a floor: the tree has at least this many.
2. Missed by chance	Each fruit has a record of how many passes saw it. From those records the system estimates the chance a visible fruit is detected on one pass, and from that, how many visible fruit were missed on every pass. With four passes and a 60% chance per pass, about 3% are missed; at 40% per pass, about 13%.
3. Never visible	Fruit buried behind other fruit cannot be estimated from images at all. On about 18 hand-counted trees per survey, the true count is compared with layer 2, giving a correction factor and its spread.
4. Range	The spread of the correction across the hand-counted trees is carried into each tree's estimate, producing a range, not just one number.
5. Orchard total	If the fruit pass covered a sample, trees are grouped by crown size from the tree map, and each group's average is scaled by the number of trees in it.
6. Check	On trees with known counts, the share of true counts falling inside the stated range is measured. A "90% range" must hold about 90% of the time or the model is rejected.

> **What the published work warns about.** A correction model that scored almost perfectly in one season did no better than a plain factor the next season. Across nine mango orchards, calibrated counts were sometimes worse than raw counts (17% against 11% average error). So the correction is refitted for every survey, kept simple, and the raw count is always shown beside it.

7. Simulation and the virtual flight

The simulator is not an animation. It is a second kind of drone behind the same interface, flying over an orchard whose true contents are known.

Three levels, built in order

Level	What it is	What it proves
Built-in simulator	A small physics model inside the backend. Runs instantly and gives the same result every time.	The mission engine, flight service, live view and data pipeline work.
Virtual orchard	Generated trees and fruit with a 3D view and a camera view in the browser.	Tree mapping, association and counting give the right answer on known truth.
Software autopilot (SITL)	The real autopilot program running on the laptop instead of on the drone.	The mission, fence and failsafes behave on the real autopilot exactly as planned.

How the virtual drone moves

Twenty times a second the simulator does the following.

1. Reads the current mission item (for example, go to this point at this height and speed).
2. Turns toward the target at a limited turn rate and accelerates at a limited rate, so corners and climbs take realistic time.
3. Updates position, height, speed, heading and tilt.
4. Drains the battery according to time, speed and climbing.
5. Optionally adds GPS error and wind drift.
6. Fires the camera when the trigger distance has been covered and writes a capture record.
7. Marks the item as reached when close enough and moves to the next.
8. Sends telemetry through the same interface a real drone uses.

It also obeys commands (pause, resume, return, land) and applies the uploaded failsafes: for example, it returns home when the simulated battery crosses the critical threshold.

How the virtual orchard is made

- **Trees:** placed in rows from the crop's spacing, with small random offsets, a few gaps, and varied height and crown size. Each has an ID.
- **Fruit:** each tree gets a fruit count drawn from a realistic spread. Each fruit gets a height on the trunk and an angle around it, in a spiral, with a size.
- **Truth table:** the simulator stores every tree and every fruit. For each camera frame it works out which fruit are in view and not blocked by the trunk, another fruit or a leaf.

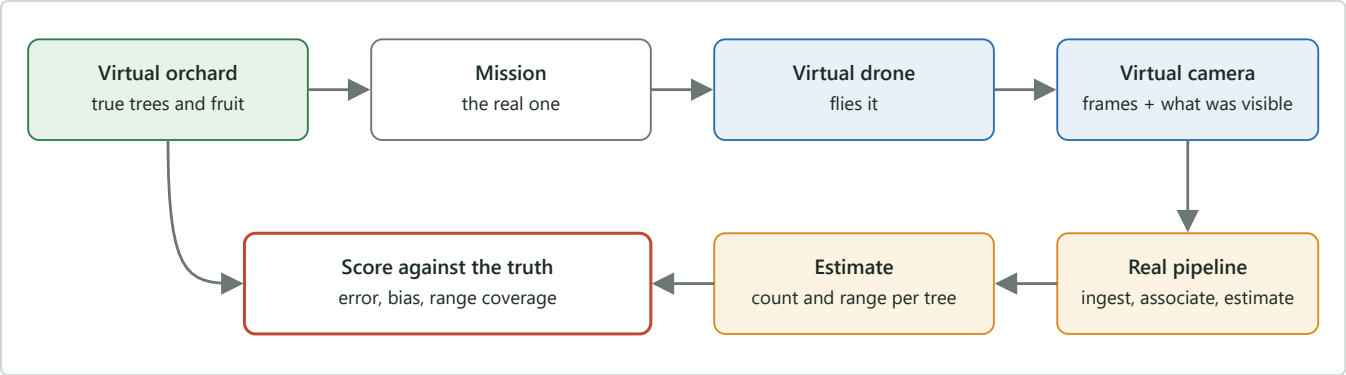


Figure 9. The simulation loop. Because the orchard's truth is known, every run ends in a score.

Two ways to produce detections in simulation

- **Synthetic detector.** Takes the list of truly visible fruit and deliberately misses some, shifts some and invents a few false ones, at rates we set. This tests association and counting under controlled error, and is the main tool.
- **Rendered frames through the real model.** The browser renders what the virtual camera sees and sends it through real ingestion. This tests the plumbing. The real model will perform poorly on drawn trees, and that is expected.

> **What simulation can and cannot tell us.** It can prove the mission flies, the data flows, fruit are merged correctly, and the stated range is honest. It cannot prove the fruit model works on real papaya, or that the real fruit pass can see the fruit. Those need real footage, which is what Phase 0 collects.

8. Step-by-step development

Each phase ends at the same gate: build, test, demonstrate, document, commit, freeze. A phase is not finished because the code runs; it is finished when its test passes.

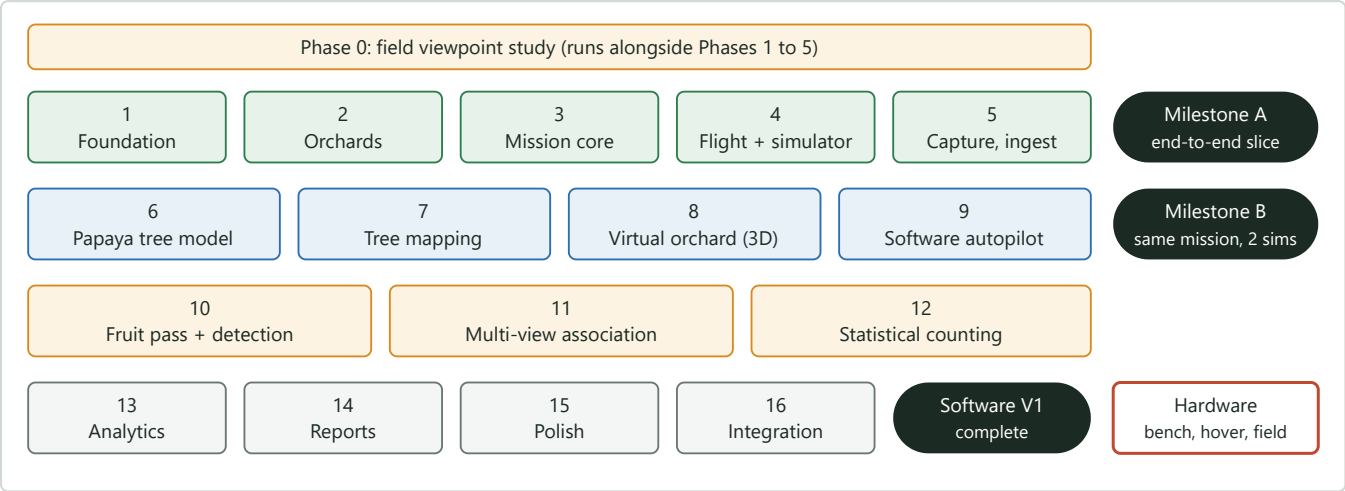


Figure 10. The order of work.

Phase	What we build	How it works	Done when
0 Viewpoint study (field)	Footage of 15 to 20 fruiting papaya trees from a camera on a pole, at several heights, distances, tilts and sides; a hand count of each tree	The pole stands in for the drone. For each viewpoint we count how many of the tree's fruit can be seen. The footage is also labelled and becomes the first training and test set for the fruit model.	A table of fruit visible per viewpoint; the fruit-pass geometry is chosen from it
1 Foundation	Project skeleton, backend, browser shell, database, live channel, logging, model registry, coordinate module, the VehicleLink interface	FastAPI backend, React and TypeScript front end, PostgreSQL with PostGIS in Docker. The coordinate module converts between latitude and longitude and local metres in one place. The registry reads crop profiles.	Health check passes, a live message round-trips, the registry loads a stub model, coordinate tests pass
2 Orchards	Create, list and open orchards; draw the boundary; pick the crop	The boundary is drawn on a MapLibre map and stored as a polygon. Area is computed by the database. The crop links the orchard to its profile.	A papaya orchard exists with a boundary and the right area
3 Mission core	Mission file format, survey generator, validation, planner screen	Intent is compiled into a flat item list by a deterministic compiler. Validation runs on the items. Every saved change makes a new revision.	A survey over the orchard gives the expected line spacing, photo count, time and battery
4 Flight	Built-in simulator, flight service, live flight screen, flight records	The simulator sits behind VehicleLink and flies the item list as described in section 7. The flight service relays telemetry to the browser and commands to the drone.	The mission flies in simulation; the track follows the plan; pause and return work

5 Capture and ingestion	Capture records, photo and video ingestion, quality checks, job queue and worker	Each trigger writes a capture record. Ingestion pairs files with records, rejects blurred or dark frames, stores the rest and queues a job. A separate worker process runs jobs.	Every trigger yields a stored capture and a finished job
Milestone A	One thin slice through everything so far	Orchard, mission, simulated flight, captures, and a tree map drawn from the simulator's truth.	The contracts are reviewed once more, then frozen
6 Papaya tree model	The existing papaya model behind the registry; a results screen	The worker loads the model named by the crop profile, runs it on overhead photos and stores crown outlines with the model version.	Real photos give stored, visible tree outlines
7 Tree mapping	Ground positions, permanent tree IDs, rows, tree map, tree page; stitched map and height model giving tree height and crown width	Each crown's pixel position is projected to the ground using the capture pose. Detections of one tree in overlapping photos are merged. A new survey is aligned to the stored map by matching the row pattern, then each tree keeps its ID.	A simulated survey recovers the virtual trees within a set position error
8 Virtual orchard	3D scene, generated trees and fruit, camera view, truth table	Three.js draws the orchard in local metres. The backend owns the truth; the browser only draws it.	The camera view matches the computed footprint
9 Software autopilot	MAVLink link, upload and read-back, failsafe configuration	The real autopilot program runs in Docker. The backend speaks MAVLink to it exactly as it later will to the Pixhawk.	The Phase 3 mission flies unchanged on the autopilot
10 Fruit pass	Fruit-mission generator driven by the crop's viewpoint template, sampled and full coverage, video ingestion with per-frame pose, fruit detection	The generator lays paths along row gaps from the tree map using the Phase 0 geometry. Frames get poses from the log. The fruit model runs on each frame.	Each chosen tree is filmed as planned; detection is scored on Phase 0 footage
11 Association	3D placement and merging of fruit; shape completion as a supporting step	Detections are linked frame to frame, placed in 3D where sight lines cross, assigned to the nearest trunk, and merged across the two sides.	Double-count and missed-count rates meet target against simulator truth and Phase 0 counts
12 Counting	The layered estimate of section 6, the calibration workflow, sampling scale-up	As described in section 6.	Stated ranges hold at the stated rate in simulation and on Phase 0 trees
13 Analytics	Orchard, tree and fruit statistics, coverage, maps, tree ranking	Queries over stored estimates, drawn as maps and charts. No health or disease advice in V1.	Numbers match the stored records
14 Reports	PDF, CSV and JSON exports; tree and fruit inventories; mission files	Generated from the same queries as the analytics screens.	A survey report opens and matches the dashboard

15 Polish	Visual effects, transitions, loading states, responsive layout	Only after function works, so effects never stand in for engineering.	Review on laptop and tablet
16 Integration	One full run from new orchard to report	All parts together, on both simulators.	Software V1 complete
Hardware	Pixhawk, on-board computer, camera	Bench test with propellers off, then a hover test, then a field survey. Only the connection address changes in the software.	A real survey produces a tree map and counts

9. Things noticed while planning

1. **Filming every tree takes hours.** About 2.2 to 4.5 hours of flight for two hectares. Sampled coverage is now a first-class mode, and the statistics scale a sample to the orchard.
2. **Low flight between rows needs precise positioning.** Standard GPS is off by about two metres, and rows are two to three metres apart. Autonomous low flight therefore needs RTK positioning, or the fruit pass uses a safer viewpoint (higher, angled down from above the next gap), or a person carries the camera. Phase 0 tests all three viewpoints.
3. **Unripe papaya are green on a green background.** Detection is harder than for red apples or orange citrus and will rely on shape and texture. Expect more labelled data to be needed.
4. **Shape completion does not count hidden fruit.** It improves size and matching. It is kept, but as a supporting step.
5. **Repeat-view statistics miss buried fruit.** Fruit hidden from every direction never appear in any view, so only hand counts can correct for them. Both layers are needed.
6. **We need our own labelled footage.** No public dataset shows papaya fruit from a drone. Phase 0 footage is the first dataset.
7. **Propeller wash at close range moves the leaves.** This may blur frames or may uncover fruit. It can only be judged with a real drone, so it is a hardware-stage check.
8. **OrchardVision does not replace QGroundControl for drone setup.** Calibration and tuning stay there.
9. **Each new crop needs its own field study.** The software is ready for many crops; the capture geometry is not transferable.

10. Decisions still open

Decision	Options and recommendation	Needed by
Orchard access for Phase 0	15 to 20 fruiting papaya trees for about a day, a camera on a pole, hand counting	As early as possible
Development database	PostgreSQL with PostGIS in Docker (recommended) or SQLite for now	Phase 1
Existing papaya model and data	Where they are stored, and whether the images are overhead or side views	Phase 6
Autopilot firmware	ArduPilot (recommended) or PX4	Phase 9
Positioning and camera	RTK or standard GPS; fixed camera or gimbal; one side camera or two	Before hardware; affects the fruit pass
Second crop for the paper test	Mango (recommended), orange or apple, written up as a profile only	Milestone A
Fruit-pass coverage	Sampled (recommended for whole orchards) or full (small blocks)	Phase 10